ADVANCED AI SOCIETY · PROOF-OF-CONTROL INITIATIVE
Working draft v0.1 · August 2026 · for co-author review prior to submission

From Message Authorization to Effect Binding: Conditions, Counterexamples, and a Conformance Harness

— Signing the bytes is not signing the deed.

Jim Schwoebel[1,2]

[1] Quome [2] Advanced AI Society, `https://advancedaisociety.org`[1]

ABSTRACT

Here is something that sounds like it must be true, and isn't. You give a program permission to do a thing. It signs a message saying what it wants. The message arrives with the signature intact, nobody tampered with anything. So the thing that happens is the thing you approved—right?

No. And it fails even when nobody is attacking you. Take a network retry over an operation that isn't idempotent. Retry with the same one-time nonce and the endpoint tells your client the operation failed—a lie, because it already committed. Retry with a fresh nonce and it does the whole thing twice. Both branches are wrong and there is no third branch. Every component followed its manual. You cannot fix it by improving the message, because on the wire an attacker replaying you and a load balancer retrying you are the same event; the difference is somebody's intent, and intent is not a field in a JSON object.

So we ask what would have to be true for the approved message to determine the effect, and give four conditions: put every input in the message, give one meaning one spelling, make redelivery harmless, and name the state you are assuming. Together they are enough. We are loud about not having shown them necessary—the document we inherited claims both directions and established one.

Then we built it and tried to break it. There is a JSON decoder you can dial to behave like Go, Node, Python, Ruby or Elixir, checked against the real runtimes rather than asserted. There is an endpoint built to be bad in five separate ways you can switch on one at a time. And there is a proxy for real APIs, which closes one condition outright, closes another only if you configure it, and cannot close the third at all—not "doesn't yet," cannot. We wrote a test asserting that third attack *succeeds*, so the claim cannot quietly grow later.

Finally we recounted a 24-endpoint survey we had inherited and the table does not say what the prose around it says: 4 / 7 / 13, not 4 / 6 / 14, one row dropped from one bucket and absorbed into another. We report that rather than quietly fixing it. We also say what the corrected number still does not license: two dozen endpoints picked for industry coverage is not a random sample, so 54% is no more a fact about the world's APIs than 58% was.

[1] Prepared within the Proof-of-Control Initiative of the Advanced AI Society. Authorship is limited to contributors to this manuscript. Co-authors will be added only upon explicit consent and substantive contribution; the author list is deliberately short at this stage rather than inherited from a sibling paper.

1 The Problem

I want to start with a case where nobody does anything wrong. That is important. It is easy to look at a security paper, find the villain, and conclude that the lesson is "don't be that guy." Here there is no villain. Everybody follows the manual and the result is still broken, and that tells you the trouble is in the arrangement rather than in anyone's carelessness.

So: a program holds a token that lets it place an order. The token is narrow, it expires quickly, and it carries a number that may be used exactly once—a nonce—so that nobody can record the request and send it again later. The program signs the bytes of its request. The endpoint checks the signature: good. It checks the nonce: never seen before, good. It writes the order into the database. It starts composing a reply.

The connection drops.

Now something between the two machines—a load balancer, a service mesh, a client library with a perfectly sensible default—decides to try again. And here is the whole paper in one paragraph: it has two ways to try again, and both of them are wrong.

Suppose it sends the identical request, same nonce and all. The endpoint looks at the nonce, sees that it has already been used, and refuses. Now, that is the correct behaviour! That is precisely what you built the nonce for. If somebody had recorded your request and was replaying it, you would want exactly this. But nobody recorded anything. The order is sitting in the database. Your program has now been told, in a well-formed and correctly signed message, that the operation failed. It didn't fail. It happened.

All right, suppose instead the program gets a fresh nonce and tries again. Now the endpoint sees a request that is legitimate, correctly signed, and has never been seen before—because it hasn't—so it does what you told it to do and places the order. Two orders. One customer. One intention.

Look at what we have. The signature check: right. The nonce check: right, and doing its job well. The retry: right, that's what reliable transports are *for*. Every piece is correct and the assembly is wrong. And you cannot repair it by making the message fancier, because the message layer is blind to the distinction that matters. To an attacker replaying you and to a load balancer retrying you, the bytes on the wire are identical. They are the same event. The difference lives in somebody's intention, and intention is not a field you can add to a JSON object.

That is the whole shape of the thing. Let us give the property we want a name: *effect binding*, meaning that the message you approved determines the effect that happens. What the retry case shows is that message authorization—which is what capability protocols give you, and they give it to you well—is a different property, and no amount of strengthening the message will turn one into the other.

There is a repair, and it is worth showing now because every repair in this paper has the same shape. The mistake is that we used one number for two jobs. The nonce is supposed to answer "is this request fresh?" and we quietly asked it to also answer "is this the same order?" Those are different questions. So use two numbers. Let the nonce change on every retry, as it should. Let the client also carry a key that names *the order*, and hold that key constant across retries. Then the fresh-nonce retry shows up carrying an order key the endpoint has seen before, and the endpoint hands back the result it computed the first time instead of doing the work again. ISO 20022 has had this for years—they call it the end-to-end identifier [10]—and HTTP has it as the `Idempotency-Key` header.

That closes one of the two branches. It does not close the other, and I want to be honest about this rather than let you discover it later. If the first reply never arrives, your program still does not know whether the effect happened. It can ask, of course. But asking is another message, and that message can be lost too, and so can the answer. You can push the uncertainty around but you

cannot make it go away. What is left over, after every fix an endpoint can make, is a committed operation that the client believes failed.

We report that leftover separately from everything else the tool finds. A well-behaved endpoint shows no findings, and if we printed "no findings" and stopped, we would be telling you the problem was gone. It isn't gone. So the tool prints the leftover under its own heading, every time, even on a clean run.

1.1 Now suppose somebody is attacking you

That was what goes wrong with nobody attacking. Here is what goes wrong when somebody is, and it is more embarrassing.

Two programs both read JSON. One decides whether the request is allowed; the other carries it out. In a real deployment they are usually written in different languages, because different teams built them. Look at this:

```
{"to":"acct-1","amount":100,"to":"acct-9"}
```

The key `to` is in there twice. Is that legal? RFC 8259 says yes and declines to say which one wins. RFC 7493 says no, don't do that [5, 4]. So the standards disagree, which means the implementations disagree, and—this is the bad part—they disagree without complaining. Go's `encoding/json` keeps the last one it sees. Erlang/OTP's `json` module, which is what Elixir uses, keeps the first one. Neither is a bug. Both are reasonable ways to read a document whose own specification shrugged.

So now: the program that decides policy reads "pay `acct-1`", thinks that looks fine, and signs. The program that moves the money reads "pay `acct-9`" and pays `acct-9`. The signature verifies perfectly. Not one byte was altered anywhere. Both programs did exactly what their documentation promised. And your money went to the wrong place.

People call this a parser differential [11]. The name doesn't matter much. What matters is that it is an old family—HTTP request smuggling is the same trick played on a front-end and a back-end that disagree about where one request stops and the next starts [14, 12]. What is new is the arrangement. When a person clicks a button, the thing that shows the button and the thing that handles the click are usually the same program, so there is nothing to disagree with. When a program signs a request on your behalf, the signer and the doer are almost never the same program. We have taken a fifty-year-old class of bug and built an industry on top of the conditions that produce it.

2 What Others Have Built

None of this is new territory. Smart people have been working on the surrounding problems for fifty years, and I want to go through what they built and be careful about what each thing does and does not settle. Being careless here is how you end up thinking a problem is solved because something nearby is solved.

Complete mediation, and the assumption hiding inside it. Saltzer and Schroeder put complete mediation near the top of their list of design principles: check every access to every object, every time [17]. The reference monitor that formalizes this [1] carries an assumption so obvious in 1972 that nobody wrote it down, which is that the monitor and the thing being protected are the same machine. The monitor looks at the request; the machine performs it. There is no gap between deciding and doing because there is no distance between them.

Now take that monitor and split it across a network. The decision happens in one process, on one host, in one programming language. The effect happens somewhere else, in a different language, written by different people. The gap that the 1972 model defined out of existence is precisely the gap I have been describing. Complete mediation isn't wrong. It simply has nothing to say about the case where the decider and the doer are two programs that have to agree on what a request means—and, as we have just seen, sometimes don't.

Capabilities. This runs from Dennis and Van Horn [6] through Levy's survey [13] to macaroons [2], which let a holder narrow a token but never widen it. That is a genuinely good answer to the question "what is this bearer allowed to ask for?" It is not an answer to "what will the endpoint do with what it asked?" The caveat language has no words for effects. It could not have; that is not what it is for.

Proofs about messages, which are proofs about messages. The Dolev–Yao model [7] and the tools built on it—ProVerif [3], Tamarin [15]—are excellent. I mean that. If you want to know what an attacker can do to your protocol messages, these are the right instruments and they are far better than intuition.

But I want to be careful, because a green checkmark from a good tool is easy to overread. When Tamarin tells you a capability protocol is secure, it is telling you something about messages: which ones an attacker can manufacture, which secrets stay secret, who agrees with whom about what. The endpoint's parser is not in the model. Its defaulting behaviour is not in the model. Its database is not in the model. And the retry case from Section 1 is completely invisible to such a proof, because both branches are protocol-legal. Nothing was forged. The proof is correct and the system is broken, and those two facts sit together without contradiction because they are about different things.

The confused deputy. Hardy's compiler [9] was talked into using its own authority for a caller who didn't have any. Same skeleton as ours: a component with power acts on a request whose meaning it got wrong. The difference is which part it got wrong. Hardy's deputy was confused about *whose* authority was in play. Ours is confused about *what was asked*.

Canonical form. RFC 8785 defines a canonical way to write JSON down [16], and RFC 7493 defines the subset of JSON on which that is even meaningful [4]. Together they are the best available answer to the duplicate-key problem, and one of our practical conclusions is dull enough to state in a sentence: decode strictly and canonicalize *before* you check the signature, not after. It is the cheapest fix in this paper and probably the one with the largest effect.

RFC 9110. HTTP already says which of its methods are idempotent and which are not [8]. `PUT` is; `POST` isn't. This is normative text, not folklore, and we lean on it directly in condition C3.

ISO 20022, which mostly solved this in the 2000s. Of everything in this section, this [10] is the strongest existing answer, and it deserves reading rather than citing. Look at what the `pain.001` credit transfer actually does. The schema is mandatory and closed, so there are no optional fields for a server to helpfully fill in on your behalf. It carries an end-to-end identifier that is separate from anything the transport does about retries. And that identifier is required, not optional, so a client cannot forget it.

That is three of our four conditions, met by a message format designed by people who could not afford to get this wrong, roughly two decades before anybody was worried about software agents. So the conditions are achievable. That is worth knowing, and it is the main reason I do not think this paper is describing an impossibility.

It is also worth noticing what ISO 20022 is not. It is not a general-purpose web API. Nobody designs a REST endpoint that way on a Tuesday afternoon, and the interesting question is how much of that discipline an ordinary API can adopt without becoming a payments network.

3 Writing It Down Precisely

Now I have to write down some symbols, and I want to say why before I do it. The symbols are not going to make anything truer than it was in Section 1. Everything important has already been said in English. What the notation buys us is that it makes it much harder to be vague—in particular, it forces us to write down exactly which things the effect is allowed to depend on, and that turns out to be the entire question. If you find the algebra unhelpful, skip to the four conditions; you will lose nothing but precision.

Think of an endpoint as a machine that takes a state and some bytes and gives you back a new state and something to look at. Let Σ be the space of states the endpoint can be in, W the byte strings that can arrive on the wire, and O the things you can observe coming back. Parsing turns bytes into a document, and it is a *partial* function $P : W \rightharpoonup D$ because plenty of byte strings are not documents at all. Then the endpoint is

$$E \;:\; \Sigma \times W \;\to\; \Sigma \times O.$$

Split that into two steps, $E(\sigma, w) = X(\sigma, P(w))$: first read the bytes, then act on what you read. And here is the part that matters. The program that decides policy has *its own* parser, which I will call P', and a policy Π that looks at documents and approves or refuses. It signs w when $\Pi(P'(w))$ says yes.

Two parsers. P and P'. Everything in this paper is downstream of the fact that we wrote two of them and then assumed they were one.

Definition 1 (Message authorization)**.** *A deployment provides message authorization if no party can cause the endpoint to accept a wire representation w that was not signed by an authorized holder.*

Definition 2 (Effect binding)**.** *A deployment provides effect binding if, for every authorized w, the resulting state transition and output $E(\sigma, w)$ are determined by $P'(w)$—the document the policy evaluated—and by nothing else.*

These two are independent, and that independence is the entire paper. The retry case has message authorization. Nobody forged anything; every signature checked out. It does not have effect binding. If you only ever ask for the first property you will get it, and you will believe you got the second one, and nothing will tell you otherwise until the money is in the wrong account.

So what would you have to demand to get the second one? Four things. Each is pinned to a function that checks it in `src/classify.rs`, so that if you disagree with a condition you can go read the code that enforces it rather than argue with a paragraph.

C1 — Put every input in the message.

Everything that changes what happens has to be in the document that got signed. Written out:

$X(\sigma, d)$ must not read anything out of σ that the client could have said and didn't. The way this fails is so ordinary that it feels like a feature. You leave out `fee_tier`; the server looks up your account preferences and fills in "priority." The policy approved an ordinary transfer. The endpoint performed an expedited one. Nobody lied to anybody. Checked by `judge_c1`.

C2 — One meaning, one spelling.
P and P' have to agree, and each has to be injective on the meanings it accepts. That is two separate demands. Two different byte strings must not mean the same thing, and—this is the duplicate-key case—one byte string must not mean two different things to two different readers. The first half is violated by a resource you can reach as both `/v1/users/me` and `/v1/users/usr_99812`; those are one thing wearing two names, and a signature over one of them says nothing about the other. Checked by `judge_c2`.

C3 — Make redelivery harmless, with two numbers not one.
Deliver the same authorized document ten times, get one effect. And the identifier that decides "this is the same document" must be a different thing from the nonce that decides "this is fresh." That second clause is the whole repair from Section 1, and leaving it out is what produces an endpoint that is idempotent in the brochure and not in practice. Checked by `judge_c3`, which grants this for free to the methods RFC 9110 already defines as idempotent, because there the specification has done the work.

C4 — Say what state you are assuming.
If the effect depends on something that can change between the moment you sign and the moment the endpoint acts, then the document has to name the state it is counting on, and the endpoint has to refuse when that has moved. In HTTP you spell this `If-Match`. The point is not the header. The point is that "transfer $100 if the balance is still what I saw" is a different instruction from "transfer $100," and only one of them can be honoured safely by a machine acting minutes later. Checked by `judge_c4`.

4 Sufficient, and Not Known To Be Necessary

Four conditions is a satisfying number. It has the shape of a finished result, and there is a real temptation, having got there, to say "an endpoint is effect-binding *if and only if* C1 through C4 hold" and go to lunch.

I want to resist that, because we have not earned the second half.

What we have is: if all four hold, you get effect binding. That is the direction with the engineering value in it, and the argument is short. If every input that matters is in the document (C1), and the document has exactly one spelling and exactly one meaning (C2), and delivering it repeatedly cannot multiply the effect (C3), and the state it assumes is named and checked (C4), then what the endpoint does is a function of the document the policy looked at. That is Definition 2. That is the whole proof, and its shortness is a point in its favour rather than against it.

What we do not have is the converse: that an endpoint failing one of these conditions must therefore fail to be effect-binding. And it is not merely unproven, it is doubtful, and I can give you two counterexamples off the top of my head.

Put the monitor on the same machine as the resource. Now you are back in 1972, the reference-monitor assumption holds again, and there is no gap for any of C1–C4 to close. The endpoint is effect-binding and satisfies none of our message-level conditions, because it doesn't need to.

Or take an endpoint whose effect is a total function of its input over a fixed state space—no stored state that can move. It satisfies Definition 2 without pinning anything, because there is nothing to pin. C4 is vacuous there.

Neither of those is exotic. Someone will build both.

The document this work grew out of states the result as an "if and only if" while establishing one direction, and I want to name that plainly rather than inherit it quietly. It is the kind of thing a good referee finds in the first ten minutes, and it should not require a referee.

Does the distinction matter, or is this pedantry? It matters, and specifically it matters for a standard rather than for a proof. Write C1–C4 into a standard as *necessary* conditions and you have built a conformance regime that refuses to certify systems which are, in fact, doing the right thing by other means. You would be telling an engineer that their co-located monitor is insufficient. It is sufficient. Your standard is wrong.

So: sufficient set, conformance target, and the converse marked as open. If somebody proves necessity later, good—and if somebody produces a third counterexample instead, that is good too, and rather more likely.

5 How We Checked

Conditions on paper are cheap. We wanted to find out whether they survive contact with real software, so we built a harness called `transit`. It has three parts, and how they relate to each other *is* the method—so I want to go through not just what each one does but why it is built that way and what would have gone wrong otherwise.

One parser with knobs on it. We could have written the differential test by requiring five language runtimes to be installed. Then the test would pass or fail depending on what happens to be on your laptop, which is a poor property for evidence.

Instead there is one JSON parser with the disagreements turned into settings: what to do with a duplicate key, what to do with a lone surrogate, how big a number may get, whether to normalize Unicode. Dial it one way and it behaves like Go. Dial it another and it behaves like Erlang.

Now, that is a claim, not a fact. When I say "this configuration behaves like Go," I am asserting something about Go that I could be wrong about, and if I am wrong the whole differential is theatre. So we try to prove ourselves wrong. Where a runtime *is* installed, we shell out to it, push the same corpus through, and demand that the imitation and the real thing agree. Where the runtime is absent the model still runs, but every row of output carries the word `Unverified`, in every format, forever, until somebody actually checks it.

Here is a case in point, and I would rather tell you than have you find it. The famous example in this area is Go's last-key-wins against Elixir's first-key-wins. There is no Elixir on our development machine. So the acceptance test does *not* assert that pair. It asserts a disagreement between two decoders that really are present and really were written independently—a strict I-JSON decoder and `serde_json`—and it exercises first-key-wins through the Elixir model, which prints `Unverified` wherever it appears. We have since measured OTP 27 directly on a different machine, and it does keep the first key. That is a fact about that machine. It does not promote the model on this one, and the tool does not pretend otherwise.

This is fussier than it needs to be if you trust us. The point is that you shouldn't have to.

An endpoint built to be bad. The thing under test is a function: state and request go in, new state and response come out. No sockets in the interesting path, so the tests cannot flake, and the

same function is what gets served over HTTP when you want to poke it by hand.

It has five separate ways of being wrong—sloppy decoding, filling in defaults from your profile, treating two URLs as one resource, forgetting idempotency, ignoring preconditions—and each one switches independently. Turn them all off and you get what we call the *boundable profile*.

That profile is the control, and controls are the part people skip. Suppose I write a check for the defaulting bypass and it reports a finding. Have I learned something about the endpoint? Not yet! A function that returns "I found a bug" unconditionally would also report a finding, and it would be useless. So every acceptance test runs the check twice: once against the hostile configuration, where it must find the bug, and once against the boundable one, where it must find nothing. A finding in the first and silence in the second is evidence about the endpoint. A finding in both is evidence about the checker, and it means you should go fix the checker before you tell anybody anything.

Something you can actually deploy, and its ceiling. The third part is a proxy you put in front of a real API. It reads every request strictly, and—this is the important bit—forwards the *canonical* form rather than the bytes that arrived, so the endpoint behind it and whatever signed the request cannot be looking at two different spellings of one document.

Its limits are more interesting than its function, so here is the whole scoreboard against the three attacks we reproduce:

Attack	Condition	Through the proxy
Duplicate-key split	C2	Rejected, unconditionally
Parameter-defaulting bypass	C1	Rejected *only if* the route names the parameters
URI alias collapse	C2	**Passes**

Look at the third row. That is not a bug we haven't got to yet. Think about what the proxy actually sees when `GET /v1/users/me` goes past: a method, a path, and no body at all. There is nothing in there to canonicalize and nothing to compare. For the proxy to know that `me` and `usr_99812` are the same person, it would have to understand the application's identity model, at which point it is not a proxy any more, it is a second copy of the application, and now you have two of them to keep in agreement. The honest statement is that a proxy cannot close this, and no amount of engineering effort changes that.

So we wrote a test that asserts the attack *succeeds*. It is in the suite, it is green, and if somebody later makes the README claim that the proxy stops all three attacks, that test goes red. I recommend this trick generally. The claims that quietly grow in the retelling are exactly the ones nobody thought to write a test for, because who writes a test that something doesn't work?

And the second row deserves a moment too. C1 is closed only if you list the parameters for that route in the configuration. If you don't, the defaulting bypass sails straight through. That means the proxy's usefulness depends on somebody doing homework, and the natural way to do that homework is to point the classifier at the API's specification and read off which parameters it says are optional. The tool that finds the problem and the tool that fixes it are the same tool, used twice.

6 The Survey We Inherited

This work inherited a survey. Somebody had gone through 24 production APIs, classified each against C1–C4 by reading the documentation, and reported that 58.3% (14 of 24) fail outright,

25.0% (6 of 24) qualify conditionally, and 16.7% (4 of 24) qualify. Those percentages are quotable, and they had already been quoted.

We put the table into a data file so the tool could count it, because we had a rule that no number gets typed into this paper by hand. The tool counted 4, 7, and 13.

That is not what the prose said. So one of them is wrong, and the useful question is which.

It's the prose. Square's `POST /v2/payments` is sitting right there in the table as a conditional row, and the 25% figure doesn't include it, and the 58% figure has swallowed it. The rows were fine all along. Somebody counted them by hand once, and the count went into the text, and after that everyone including us quoted the text instead of recounting the rows. An earlier draft of this very paper repeated the published figures.

I want to dwell on this for a moment, because the arithmetic is trivial and the lesson isn't. We did not find this by being clever. We found it because we had made it impossible to write the number down without computing it, and the computation disagreed. The rule cost almost nothing to adopt and it caught something on its first application. That is a better argument for the rule than any amount of methodology talk.

And now the part that the corrected number does not entitle anyone to skip. Twenty-four endpoints chosen to cover a spread of industries is not a random sample of anything. It was assembled for coverage, which is the right way to assemble a set of illustrations and the wrong way to assemble a basis for percentages. Worse, every one of these classifications came from reading documentation, and two of our four conditions are not visible in documentation even in principle. No specification will tell you what a server quietly fills in from your account settings. No specification will tell you whether an effect depends on state that moved while you weren't looking. Those are facts about running code.

So 54% is not a fact about the world's APIs. Neither was 58%. What we have is a taxonomy with two dozen worked examples, which is genuinely useful and is not a measurement, and the difference between those two things is most of what this section is about.

Because that distinction is easy to lose in transit, every rendering of this table carries a banner saying it is a hypothesis—including the generated LaTeX, where the banner is the caption. You cannot `\input` the table into a document without the caveat coming along with it. We did that deliberately, after noticing how easily we had ourselves absorbed the original percentages.

7 Does It Actually Work?

Every number in this section is generated. None of it is typed. The tables below are written by `scripts/regen-results.sh`, the build regenerates them and fails if they have drifted from the code, and the reason for all that machinery is in Section 6: we tried the other way and it put a wrong percentage into circulation.

7.1 Where decoders disagree

Specification	Verdict	Failing conditions
`iso20022-pain001`	Qualified	none
`s3-put-if-match`	Qualified	none
`slack-chat-postmessage`	NonQualifying	C1, C2, C3
`stripe-charges`	Conditional	none

Table 1: HYPOTHESIS, NOT DATA. These 24 classifications are derived from published documentation and have not been measured against a running endpoint. C1 and C4 are properties of an implementation, which no document reveals. The sample was chosen for sector coverage, so it is a convenience sample and the shares below are not a population estimate. Verifying these rows by live probing is work this repository has not done.

Sector	Provider	Endpoint	C1	C2	C3	C4	Hypothesis
Financial / Payments	ISO 20022	`pain.001 credit transfer`	S	S	S	S	Qualified
Financial / Payments	Stripe	`POST /v1/charges`	CD	S	CD	F	Conditional
Financial / Payments	Square	`POST /v2/payments`	CD	S	CD	F	Conditional
Financial / Payments	Plaid	`POST /link/token/create`	F	S	F	F	NonQualifying
Cloud Infrastructure	AWS EC2	`RunInstances`	F	F	CD	F	Conditional
Cloud Infrastructure	AWS S3	`PUT /object`	F	F	S	S	Qualified
Cloud Infrastructure	GCP Compute	`instances.insert`	F	F	CD	F	Conditional
Cloud Infrastructure	Azure ARM	`PUT /subscriptions/...`	F	F	S	S	Conditional
Cloud Infrastructure	Cloudflare	`POST /dns_records`	F	F	F	F	NonQualifying
Developer Tools	GitHub	`POST /user/repos`	F	F	F	F	NonQualifying
Developer Tools	GitHub	`PUT /repos/{o}/{r}/contents`	S	S	S	S	Qualified
Developer Tools	GitLab	`POST /api/v4/projects`	F	F	F	F	NonQualifying
Developer Tools	Docker Engine	`POST /containers/create`	F	F	F	S	Conditional
SaaS / Productivity	Google Calendar	`POST /events`	F	F	F	F	NonQualifying
SaaS / Productivity	Google Workspace	`POST /gmail/v1/users/me/ messages/send`	F	F	F	F	NonQualifying
SaaS / Productivity	Microsoft Graph	`POST /v1.0/me/sendMail`	F	F	F	F	NonQualifying
SaaS / Productivity	Slack	`POST /chat.postMessage`	F	F	F	F	NonQualifying
SaaS / Productivity	Jira	`POST /rest/api/3/issue`	F	F	F	F	NonQualifying
SaaS / Productivity	Salesforce	`POST /sobjects/Account`	F	S	F	F	NonQualifying
SaaS / Productivity	HubSpot	`POST /crm/v3/objects/ contacts`	F	F	F	F	NonQualifying
Identity / Security	Okta	`POST /api/v1/users`	F	F	F	F	NonQualifying
Identity / Security	Auth0	`POST /api/v2/users`	F	F	F	F	NonQualifying
Database / Storage	HashiCorp Vault	`POST /v1/secret/data`	F	F	S	S	Qualified
Database / Storage	Pinecone	`POST /vectors/upsert`	F	F	S	F	Conditional

S = satisfied, CD = client-dependent, F = fails, U = unknown.

7.2 The survey is a hypothesis

8 What We Still Don't Know

This is the section I would read first if somebody handed me this paper, so I have tried to make it the honest one rather than the modest-sounding one.

We mostly cannot check the thing we are asking you to check. Start with the worst of it. Two of our four conditions—C1 and C4—are properties of running code, not of published interfaces. If you do not have the source, classifying an endpoint means inferring what the implementation does from what its documentation says and from poking at it, and the entire credibility of any survey rests on how that inference gets made.

Our classifier answers `Unknown` whenever a document cannot settle the question, and refuses to guess. That is honest. It is also not a measurement, and I do not want the honesty to be mistaken for rigour. The way to turn these rows into evidence is to probe endpoints that are actually running. We built the gate that would let somebody do that safely—it demands a written declaration of authorization, a contact address, and a command-line flag, and refuses otherwise—and then we did not run it. So the empirical half of this paper is a plan, not a result, and Section 6 is a hypothesis wearing a table.

Checking somebody who isn't helping you. Everything above assumes you may ask the endpoint questions. Suppose you may not: you are an auditor, or a customer doing diligence, and you have no account and no permission. Is there any way to establish whether a third-party API is effect-binding? We do not know of one that is both sound and lawful, and I suspect this is hard rather than merely unexplored.

What do you do about the ones that can't be fixed? Thirteen of the twenty-four endpoints in that survey cannot satisfy these conditions without changing their public interfaces, and most of them are not going to. Telling those operators "adopt C1–C4" is not advice, it is a complaint. The real answers are presumably about detecting the damage afterwards, keeping the blast radius small, and compensating controls—and we have developed none of that here.

Two calls in a row. Now for the one I think is actually the important open problem. This entire paper is about one authorized message producing one effect. That is not what agents do. An agent runs a workflow: call something, look at the answer, decide what to call next, feed the result into the one after that. Each step's input depends on the previous step's output, so the thing a user approved at the start was not a message at all—it was closer to a goal.

Does effect binding compose across such a sequence? We don't know. What would it even mean to authorize a *plan* rather than a message, when the plan's later steps are not determined until the earlier ones return? We don't know that either, and I am not sure the vocabulary exists yet.

We have said nothing about it in this paper. It is very likely the question the whole agent business actually poses, and the four conditions here are, at best, what you would need to have settled before you could begin asking it.

References

[1] James P. Anderson. Computer security technology planning study. Technical Report ESD-TR-73-51, Electronic Systems Division, Air Force Systems Command, 1972.

[2] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2014.

[3] Bruno Blanchet. An efficient cryptographic protocol verifier based on prolog rules. In *Proceedings of the 14th IEEE Computer Security Foundations Workshop (CSFW)*, pages 82–96, 2001.

[4] Tim Bray. The I-JSON message format. Technical Report RFC 7493, Internet Engineering Task Force, 2015.

[5] Tim Bray. The javascript object notation (JSON) data interchange format. Technical Report RFC 8259, Internet Engineering Task Force, 2017.

[6] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.

[7] Danny Dolev and Andrew C. Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983.

[8] Roy T. Fielding, Mark Nottingham, and Julian Reschke. HTTP semantics. Technical Report RFC 9110, Internet Engineering Task Force, 2022.

[9] Norm Hardy. The confused deputy: (or why capabilities might have been invented), 1988.

[10] International Organization for Standardization. ISO 20022: Financial services — universal financial industry message scheme. `https://www.iso20022.org/`, 2013. The `pain.001` customer credit transfer initiation message is the case discussed here.

[11] Dan Kaminsky, Meredith L. Patterson, and Len Sassaman. PKI layer cake: New collision attacks against the global X.509 infrastructure. Financial Cryptography and Data Security, 2010. The canonical statement of the parser-differential family.

[12] James Kettle. HTTP desync attacks: Request smuggling reborn. Black Hat USA, 2019.

[13] Henry M. Levy. *Capability-Based Computer Systems*. Digital Press, 1984.

[14] Chaim Linhart, Amit Klein, Ronen Heled, and Steve Orrin. HTTP request smuggling. Watchfire white paper, 2005.

[15] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. The TAMARIN prover for the symbolic analysis of security protocols. In *Computer Aided Verification (CAV)*, volume 8044 of *Lecture Notes in Computer Science*, pages 696–701, 2013.

[16] Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON canonicalization scheme (JCS). Technical Report RFC 8785, Internet Engineering Task Force, 2020.

[17] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.